As an alternative to phonetic analysis, it is possible to express the lexicographic distance (the number of inserts, deletes, or substitutions) between two strings. The Levenshtein distance algorithm (from *Binary codes capable of correcting deletions, insertions, and reversals* – Vladimir Levenshtein, 1966) provides a method for such computation, implemented in PHP with the `levenshtein()` function:

```
var_dump(levenshtein('Hello world', 'Hello world'));
var_dump(levenshtein('Hello WORLD', 'Hello world'));
var_dump(levenshtein('status', 'stats'));
```

```
Output:
int(0)
int(5)
int(1)
```

Without going too deep into the linear algebra behind it all, the Levenshtein algorithm finds the minimum number of insertions, deletions, and substitutions required to transform one string into another. In the code above, we can see the first comparison is between identical strings, so the distance is *0*. Next, we substitute `WORLD` for `world`, so the distance is *5* (because the function is case-sensitive), and finally, the distance between `'status'` and `'stats'` is *1* (deletion). The `levenshtein()` function also accepts additional arguments to provide integer weighting for the different types of transformation:

```
$insertCost = 2;
$replaceCost = 2;
$deleteCost = 1;

var_dump(levenshtein('status', 'statu',
    $insertCost,
    $replaceCost,
    $deleteCost
));
var_dump(levenshtein('status', 'statue',
    $insertCost,
    $replaceCost,
    $deleteCost
));
```